



Serving of LLM-Based Applications: Parrot & Pie

Tejas Maire, Ajay Kumar, Allison Okimoto, Jamal Hassunizadeh



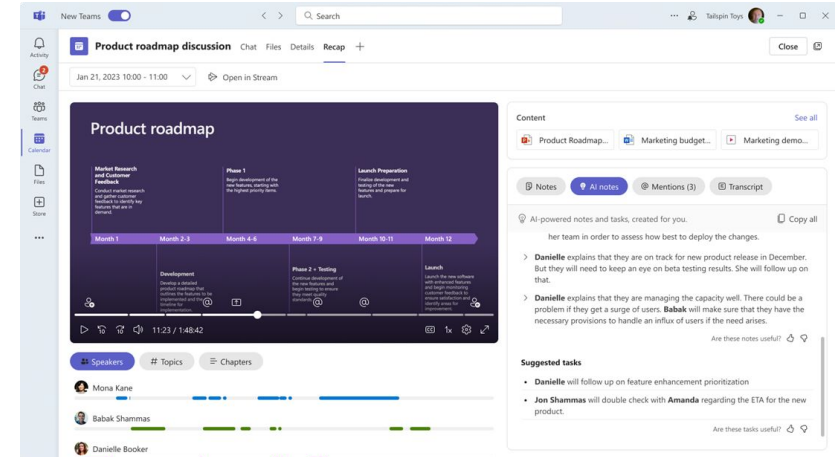
Parrot: Efficient Serving of LLM-based Applications with Semantic Variable

Background



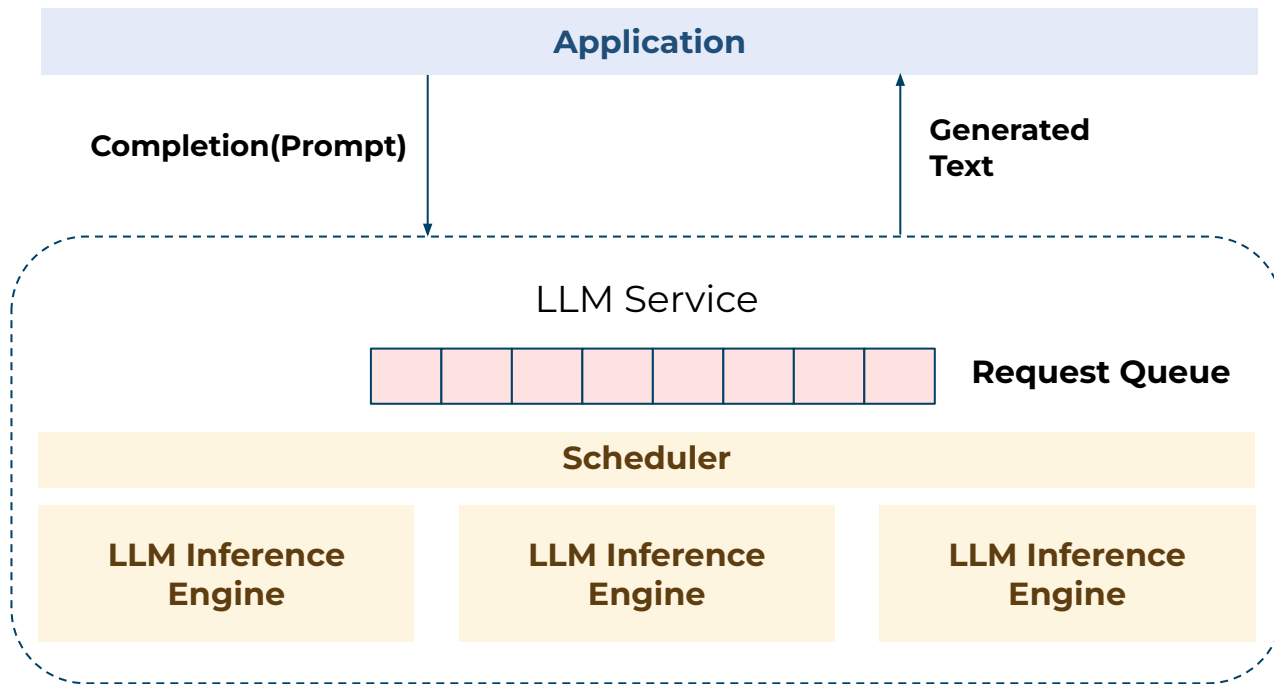
AI Agents

- Large language models (LLMs) have demonstrated a remarkable language understanding capability
- Enables a shift in application development to incorporate AI agents
- AI agents or co-pilots, communicate with LLM services via natural language to accomplish a task collaboratively





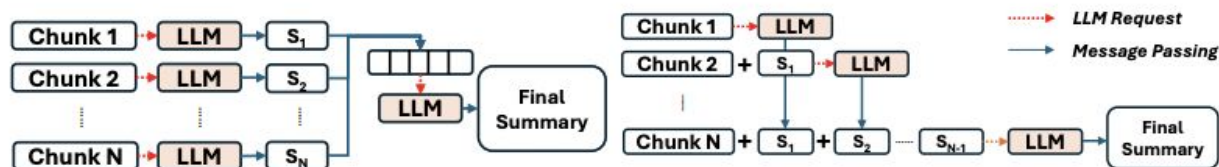
LLM Services



Most LLM services are provisioned as a conditional generation service via a text completion API.

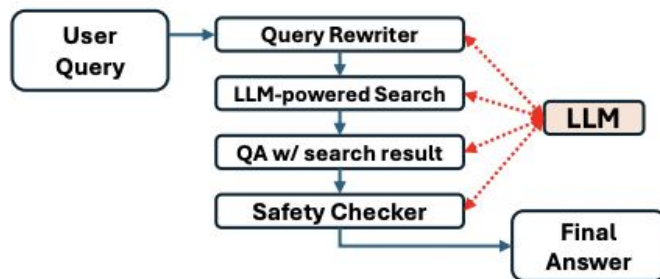


LLM-Based Applications

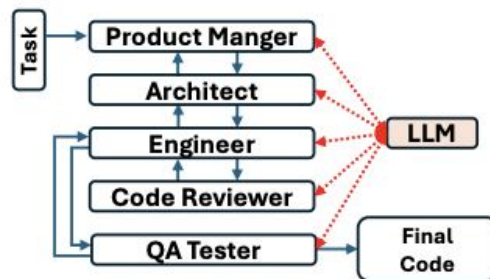


(a) Map-Reduce Summary

(b) Chain Summary



(c) LLM-Powered Search



(d) Multi-agent Coding

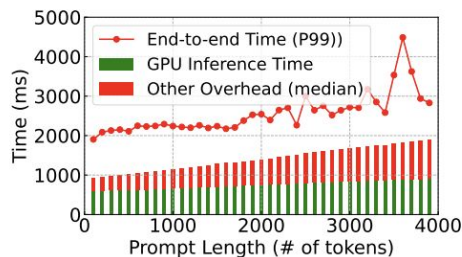


MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Problems

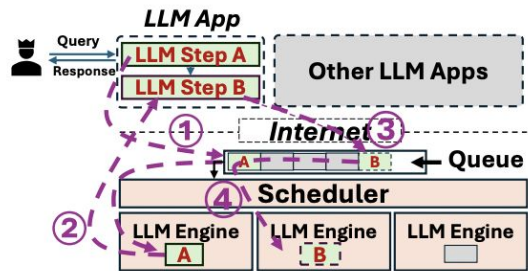
Overhead of Consecutive Requests

- LLM applications frequently make multiple LLM calls to complete a single task
 - Output of the previous request may be the direct input of the next one
- Existing LLM services serve each request **independently** without considering the dependency among such requests (no application-level knowledge)



(a) Latency Breakdown

Significant portion of the latency of LLM API call originates outside the LLM engine.

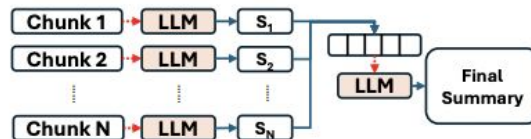
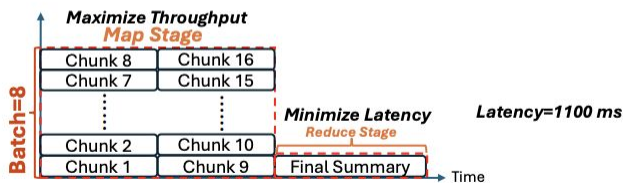
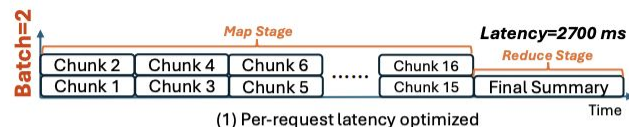


(b) Current LLM Services

Heavy network latency (2 round-trips)
Extra queuing delays

Misaligned Scheduling Objectives

- LLM services treat every request as **independent**, so they optimize per-request latency
 - Limit the number of concurrent requests running on each LLM engine to control the latency of requests (less throughput/lower batch-size)
- LLM-based applications optimize for end-to-end latency -> Mismatched Objectives!
- Challenge of managing diverse LLM requests with varying performance objectives



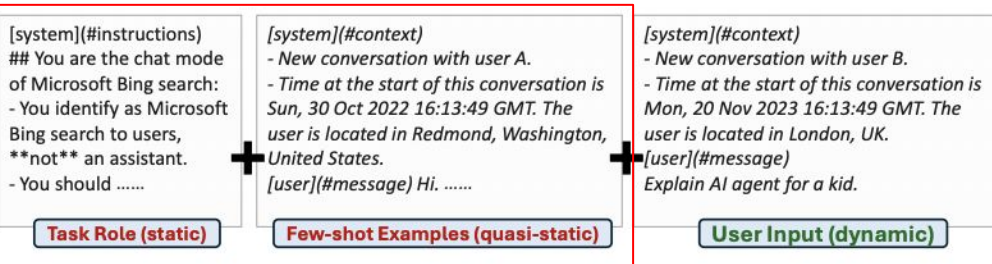
(a) Map-Reduce Summary



Redundant Computations

- LLM-based applications exhibit high redundancy in prompts
- This redundancy utilizes GPU memory bandwidth and is computed multiple times
 - Optimizations in LLM engines to avoid redundant GPU memory of shared prompt exist!
 - The request scheduler must be “smart enough” to group prompt-sharing requests together on the same engine to take advantage of these engine-level optimizations

LLM-based App.	# Calls	Tokens	Repeated (%)*
Long Doc. Analytics	2 ~ 40	3.5k ~ 80k	3%
Chat Search	2 ~ 10	5k	94%
MetaGPT [22]	14	17k	72%
AutoGen [54]	17	57k	99%





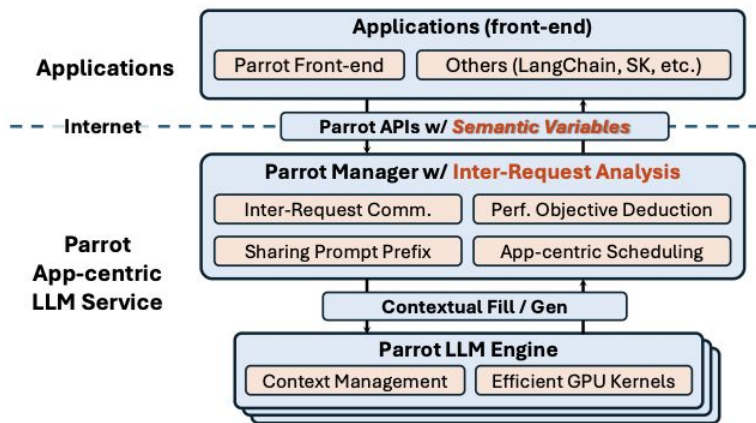
MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Solution



Parrot Design

- Introduce **Semantic Variable** to enable semantic understanding
- **Parrot Manager**
 - Receives LLM requests
 - Derives the application-level knowledge/relationships between requests
 - Optimizes end-to-end performance of application
 - Schedule requests to **LLM Engine**
 - GPU server (or group) in the cluster that can serve LLM requests independently





Semantic Variable

- Implement LLM request as an async **Semantic Function**
- Semantic Variable** is a placeholder in a prompt that represents either an input or output variable of a Semantic Function
 - preserves the prompt structure for further inter-request analysis
- Orchestration functions connect multiple semantic functions

```
import Parrot as P
from Parrot.PerformanceCriteria import LATENCY
```

```
@P.SemanticFunction
def WritePythonCode(task: P.SemanticVariable):
    """ You are an expert software engineer.
    Write python code of {{input:task}}.
    Code: {{output:code}}
    """
```

```
@P.SemanticFunction
def WriteTestCode(
    task: P.SemanticVariable,
    code: P.SemanticVariable):
    """ You are an experienced QA engineer.
    You write test code for {{input:task}}.
    Code: {{input:code}}.
    Your test code: {{output:test}}
    """
```

```
def WriteSnakeGame():
    task = P.SemanticVariable("a snake game")
    code = WritePythonCode(task)
    test = WriteTestCode(task, code)
    return code.get(perf=LATENCY), test.get(perf=LATENCY)
```

triggers
submit
API to
submit an
LLM
request

get API fetches value of semantic variable,
with annotation of performance criteria



Example

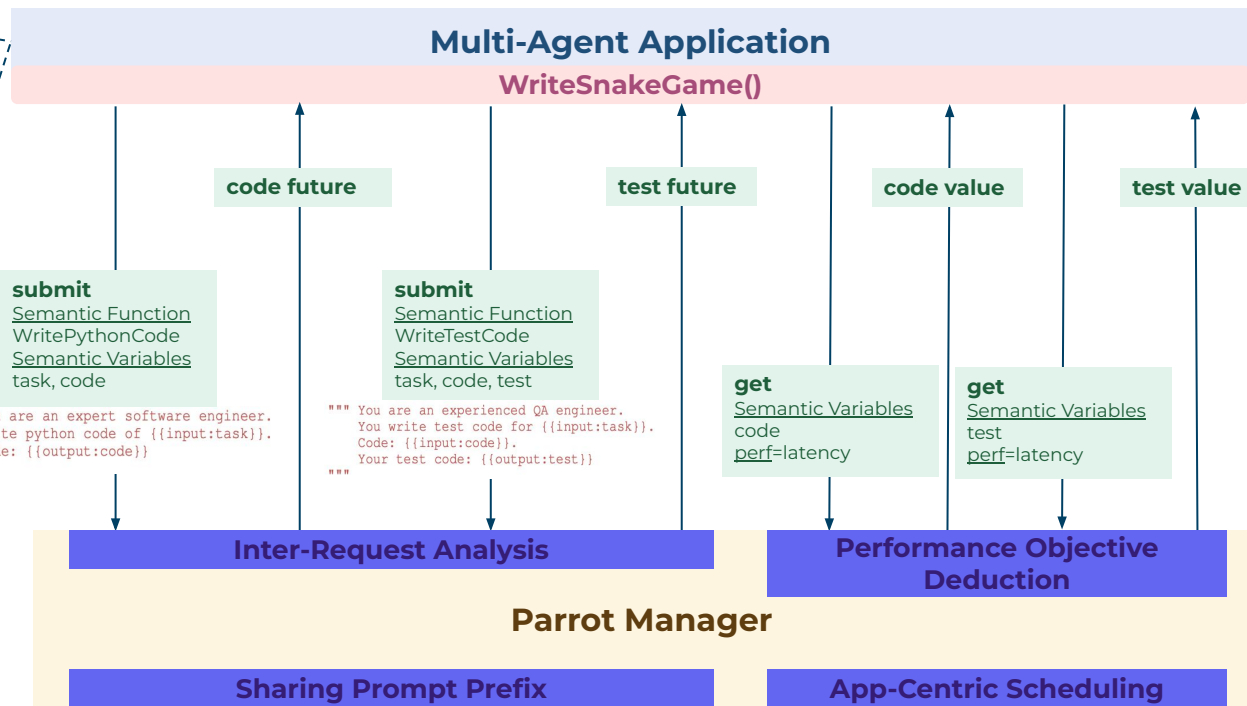
```
import Parrot as P
from Parrot.PerformanceCriteria import LATENCY

@P.SemanticFunction
def WritePythonCode(task: P.SemanticVariable):
    """ You are an expert software engineer.
    Write python code of {{input:task}}.
    Code: {{output:code}}
    """

@P.SemanticFunction
def WriteTestCode(
    task: P.SemanticVariable,
    code: P.SemanticVariable):
    """ You are an experienced QA engineer.
    You write test code for {{input:task}}.
    Code: {{input:code}}.
    Your test code: {{output:test}}
    """

def WriteSnakeGame():
    task = P.SemanticVariable("a snake game")
    code = WritePythonCode(task)
    test = WriteTestCode(task, code)
    return code.get(perf=LATENCY), test.get(perf=LATENCY)
```

Executor fires scheduled requests
to their assigned LLM Engine!





Inter-Request Analysis

1. **DAG-Based Analysis:** Maintains a DAG for each user's session
 - creates DAG by linking edges with Semantic Variables it refers through placeholders in the prompts.
 - analyze dependencies of LLM Requests (GetProducer/Consumer)
 - scheduling preference by analyzing the DAG and the performance objective
2. **Prompt structure-based analysis:** extracting hash values of an LLM request (**PrefixHash**)
 - used to detect commonality (shared prompts) across multiple requests

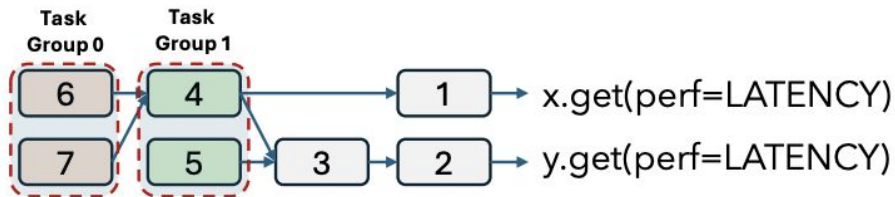




Performance Objective Deduction

- **Performance Objective Deduction:** deduce the application-level performance criteria to optimize the end-to-end performance of applications thru Semantic Variable annotations

Latency-sensitive applications:
reduce the completion time of
the entire DAG of requests



- Parrot will traverse the DAG in reverse topological order and mark requests as latency-sensitive.
- Create task groups that represent Parallel LLM requests and attempt to maximize throughput to reduce end-to-end latency



Sharing Prompt Prefix

- **Sharing Prompt Prefix** by mapping a prefix of tokens to a list of requests
 - LLM engine creates context, like KV cache, to store state of the model execution for a request
 - Parrot searches **PrefixHash** of a request's prompt in its Key-Value store to route prompt-sharing requests to same engine to utilize existing context



Parrot Key-Value Store	
Key (PrefixHash)	Value (Requests)
Hash(You are an expert software engineer. Write python code of)	Request(WritePythonCode)
Hash(You are an expert code of: {{input:task}}. Code:)	Request(WritePythonCode)

Additional Entries for [Request(**WriteTestCode**)]



Application-Centric Scheduling

- Scheduler matches LLM requests to LLM engines
- **Goal:** meet the performance goals of LLM applications while optimizing GPU cluster utilization
- **Scheduling Principles:**
 - group LLM requests with similar performance requirements
 - maximize opportunities for sharing across requests

Algorithm 1: Parrot's Request Scheduling.

Data: Q : the request queue

```
1  $Q.sort()$  ; /* Topological order */
2 for  $r \in Q$  do
3    $SharedReqsInQueue, CtxInEngine =$   
    $FindSharedPrefix(r);$ 
4   if  $r.TaskGroup \neq \emptyset$  then
5      $r^* = FindEngine(r.TaskGroup);$ 
6   else if  $SharedReqsInQueue \neq \emptyset$  then
7      $r^* = FindEngine(SharedReqsInQueue);$ 
8   else if  $CtxInEngine \neq \emptyset$  then
9      $r^* = FindEngine(r, filter=CtxInEngine);$ 
10  if  $r^* = \emptyset$  then
11     $r^* = FindEngine(r);$ 
12   $Q.remove(r^*);$ 
```

find existing prompt-sharing requests and engine contexts

allocate the entire task group together

assign common prefix requests/contexts to same LLM engine

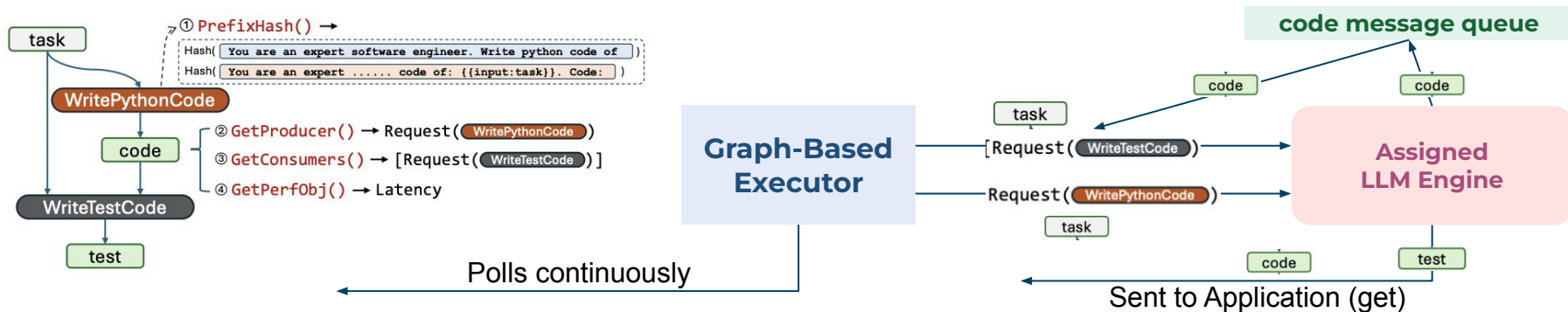
schedule independently

Schedules app requests together to prevent interleaving!



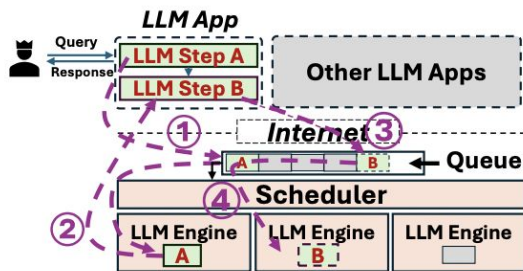
Graph-based Executor

- **Serving Dependent Requests** efficiently through a **graph-based executor**
 - executor polls continuously and fires request the instant its dependencies are resolved
 - message queues per Semantic Variable pass outputs directly to the next request -> removes need for client round-trips
 - moves the orchestration logic from the client into the LLM service itself



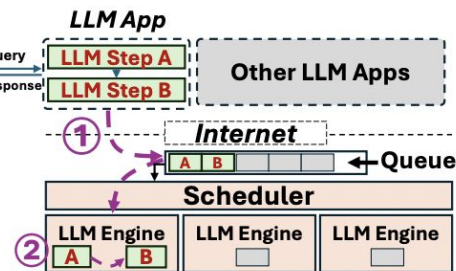


Comparison



(b) Current LLM Services

Heavy network latency (2 round-trips)
Extra queuing delays



(c) Our system: Parrot

Less back-and-forth (1 round-trip)
No queuing delay (auto fire request)
Less inference time (reuse existing LLM engine context)

Parrot optimizes for the end-to-end performance of LLM applications!



Parrot Implementation

```
(submit) {"prompt": str, "placeholders": [{"name":  
    ↳ str, "in_out": bool, "semantic_var_id": str,  
    ↳ "transforms": str}, ...], "session_id": str}  
  
(get) {"semantic_var_id": str, "criteria": str,  
    ↳ "session_id": str}
```

- **Parrot Front-end:** provides the abstraction/APIs of Semantic Variable and Semantic Function
- **Kernel Optimization**
 - **Problem:** vLLM's GPU kernel excessively reloads shared prefix tokens from global to shared memory, impeding attention score computations
 - **Solution:** developed novel GPU kernel and integrated **PagedAttention** and **FlashAttention** to accelerate attention decoding computation involving shared prefixes.
- **Universal Engine Abstraction:** minimal capability required to integrate Parrot w/ LLM engines

```
def Fill(token_ids: List[int], context_id: int,  
    ↳ parent_context_id: int)  
def Generate(sampling_configs: Dict, context_id:  
    ↳ int, parent_context_id: int)  
def FreeContext(context_id: int)
```



Differences from Related Work



	Existing Systems	Parrot
Deep Learning Model Serving Systems (Clipper, TensorFlow Serving, Clockwork, REEF, AlpaServe, Orca (iteration-level batching / continuous batching), vLLM (PagedAttention for memory efficiency))	• Serve general deep learning models • Don't consider the unique requirements of large language models • Treat requests independently	• Models LLM applications as DAGs of requests using Semantic Variables and exploits the commonality of different requests • Performs inter-request analysis and detects dependencies between requests
LLM Orchestration Frameworks (LangChain, Semantic Kernel, PromptFlow, LlamaIndex)	• Build workflows and send requests to LLM APIs • Prompt structure is lost before execution	• Exposes workflow structure to the LLM serving system • Uses semantic variables



Strengths, Weaknesses, and Future Work

Significant End-to-End Latency Reduction

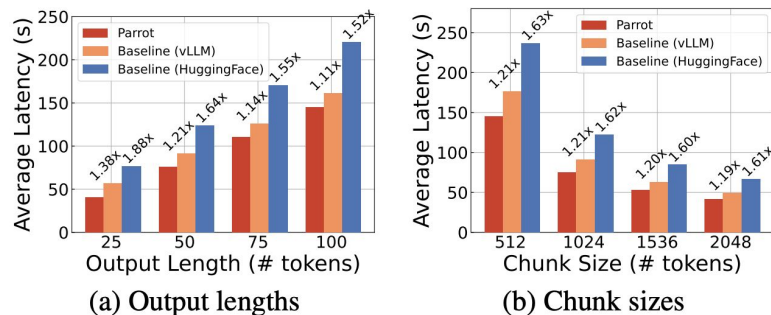


Figure 11: Average E2E latency of chain summarization with varying output lengths and chunk sizes.

****1.38x faster than vLLM and 1.88x faster than HuggingFace****

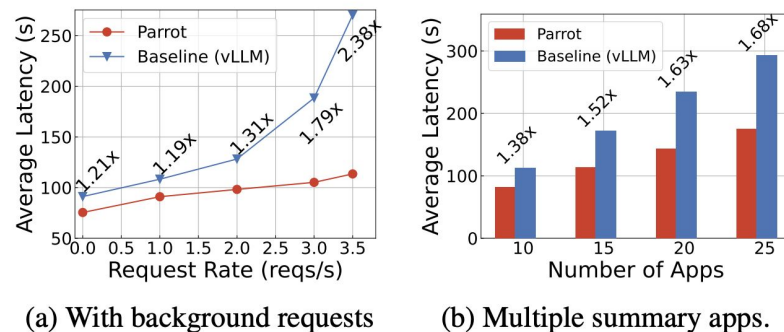


Figure 12: Average E2E latency of chain-summary with background requests or other chain-summary applications.

****2.38x lower latency when background LLM requests introduce queuing delays****



Higher Throughput for Map-Reduce Workloads

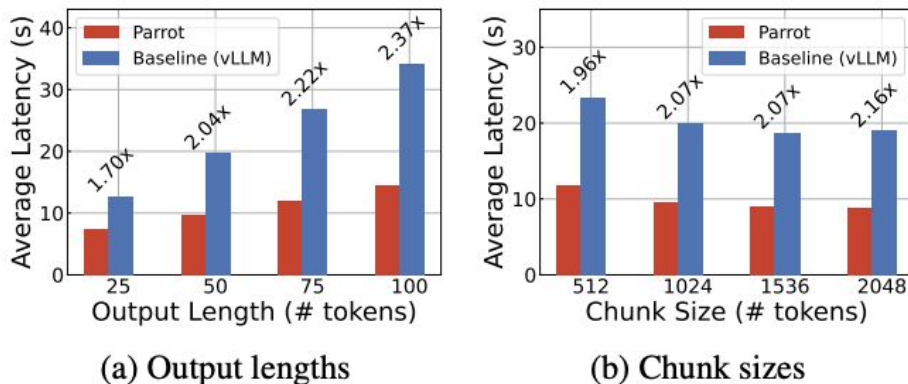


Figure 14: Average E2E latency of Map-Reduce document summary with varying output lengths and chunk sizes.

2.37x speedup over baseline systems



Efficient Prompt Prefix Sharing

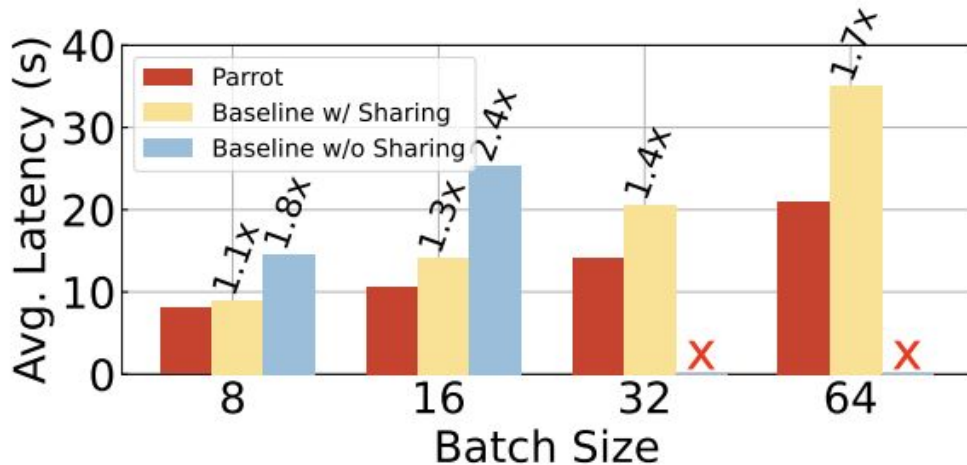
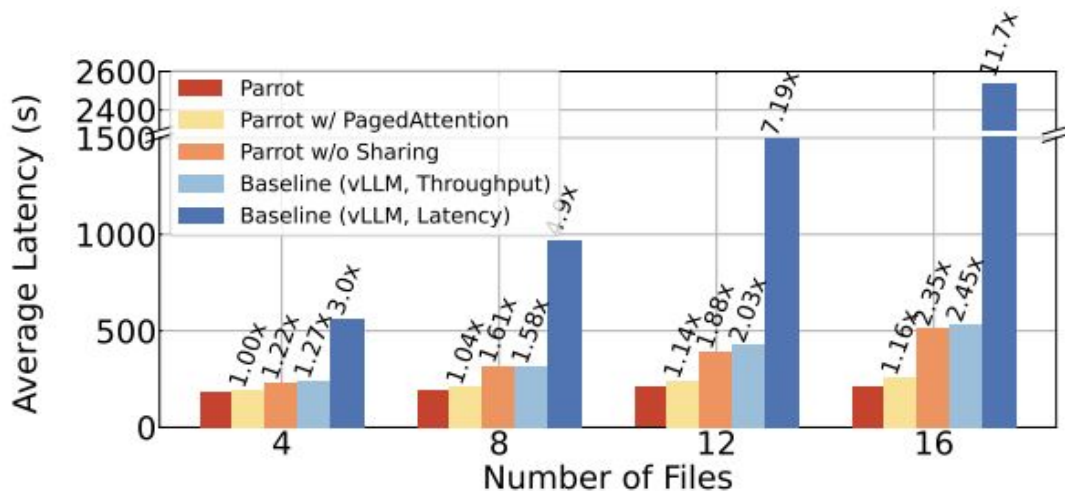


Figure 15: Latency of Bing Copilot with varying batch sizes.

1.8x-2.4x speedup compared to systems without prompt sharing. 1.1x-1.7x faster than vLLM with static prefix sharing due to improved GPU kernel optimization



Strong Performance for Multi-Agent LLM Applications



(a) End-to-end Latency

11.7x speedup over latency-centric baseline systems



Weaknesses

- Limited support for **dynamic workflows** and control flow
- **Native functions must run on the client side**, not inside the LLM service
- Assumes workflows can be represented as a **predefined DAG of requests**
- Does not yet handle **advanced scheduling challenges** such as fairness, failures, or heterogeneous clusters



Directions for Future Work

- Support **dynamic control flow** (e.g. conditional branches and loops) and enable **server-side execution of native functions**
- Explore **speculative execution** for dynamic application branches
- Extend inter-request analysis to support **fair scheduling, failure handling, and heterogeneous clusters**
- Improve integration with **LLM orchestration frameworks**



Pie: A Programmable Serving System for Emerging LLM Applications

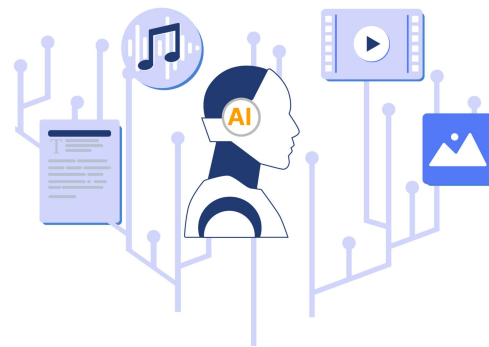


MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Background

The Modern LLM

- Large language model (LLM) usages are evolving far beyond simple text completion and must support increasingly common agentic workflows
- Require sophisticated strategies to execute LLM application logic ranging from token to workflow level
 - Handle real-time audio and visual user events
 - External tool integration
 - Code execution environment integration
- Parrot makes progress toward this, but important challenges remain





MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Problems



Requirements Identified

R1

Application-Specific KV-Cache Control

Workflows need fine-grained control over KV cache management:

- Customizing allocation
- Eviction policies
- Reuse strategies

Based on app-specific logic, not system-wide heuristics

R2

Customizable Generation Processes

Decoding methods, sampling strategies, and safety grounding need direct control over the generation pipeline.

- Token prediction loop
- Sampling loop

Per-request or even per-step basis

R3

Integrated Computation and I/O

Agentic workflows need tight coupling of operations:

- Token generation
- Arbitrary computation
- I/O operations

Without prohibitive latency or complex orchestration

Limitations of Existing LLM Serving Systems

- Implicit KV cache management → (fails R1)
- Inflexible decoding process → (fails R2)
- Poor workflow integration → (fails R3)

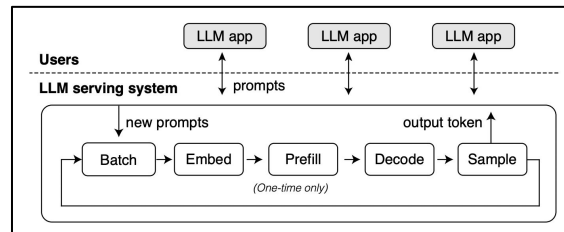


Diagram of Existing LLM Serving System Architecture

Root Cause: Monolithic Token Generation Loop (See Diagram)



MICHIGAN ENGINEERING
UNIVERSITY OF MICHIGAN

Solution



PIE: Architectural Shifts

**Dismantles the monolithic decoding loop
into fine-grained procedures**

Handlers

**Delegates end-to-end control to
user-provided programs**

Inferlets



Three-Layer Architecture

Application Layer

Inferlet

Inferlet

Inferlet

Inferlet Lifecycle Manager
(ILM)

Control Layer

Event Dispatcher

Controller

command
queues

Batch Scheduler

Inference Layer

API Handler

API Handler

API Handler

API Handler



Example Walkthrough

→ R1 (KV / control): fine-grained control via handlers
→ R2 (custom workflows): Different inferlets = different generation strategies
→ R3 (I/O integration): Yelp API call is inside the workflow

Users:

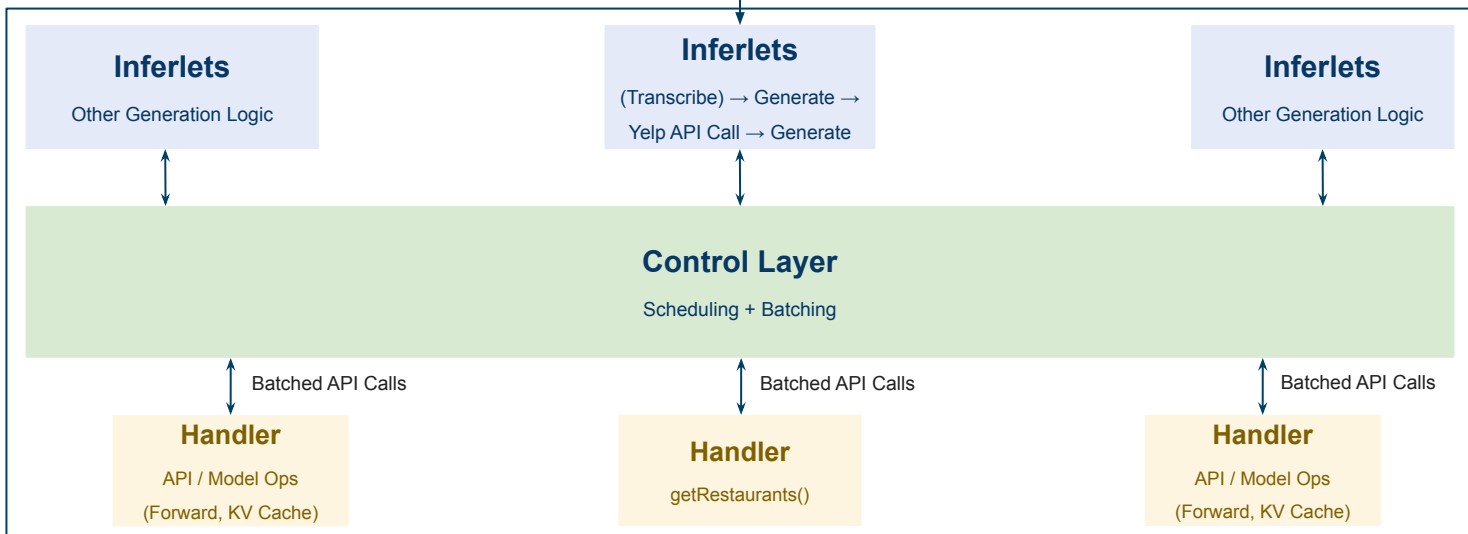
LLM App

Receive user audio query: "Find a good Italian restaurant nearby"



LLM Serving

System:





Application Layer

Application Layer

Inferlet

Inferlet

Inferlet

Inferlet Lifecycle Manager
(ILM)

- Executes users programs in a runtime that provides isolation between them
- Uses WebAssembly to sandbox instances
- Supports up to 1,000 concurrent instances



Inferlets

Inferlet = user-defined program controlling inference

Application Layer

Inferlet

Inferlet

Inferlet

Inferlet Lifecycle Manager
(ILM)

- Any language compiling to WebAssembly (Rust, C++, Python)
- Orchestrates generation by issuing API calls to Pie's handlers
- Manage resources explicitly, customize sequence with handlers, integrate computation and I/O
- Inferlet Lifecycle Manager manages creation, destruction, & communication of inferlets
- Users submit Wasm binaries and interact with running programs via send/receive APIs



Inferlet Programming Model

Pie has 42 APIs spread across three stages (organized into Traits):

Embed	Forward	Sample
Function Examples: <code>embed_txt</code> <code>embed_img</code> <code>tokenize</code> <code>detokenize</code>	Function Examples: <code>forward(ikv, iemb, okv, oemb, mask)</code> <code>forward_with_adapter</code>	Function Examples: <code>get_next_dist</code> <code>get_vocabs</code>
What it does: Prepares input embeddings from raw data (images and text)	What it does: Computes output embeddings from input embeddings, potentially utilizing the KV	What it does: Allows user to implement any sampling their application needs
Why it matters: Handles multimodal inputs; tokenize/detokenize give full text control	Why it matters: Explicit mask enables beam search, attention sink, windowed attention — no system changes	Why it matters: Inferlet controls sampling: custom schemes, watermarking, safety filters

Resources - explicitly allocated and freed by inferlet:

- Embed - a sequence of token embeddings, allocated per-token
- KvPage - a chunk of KV cache, following PagedAttention (8-32 tokens per page)



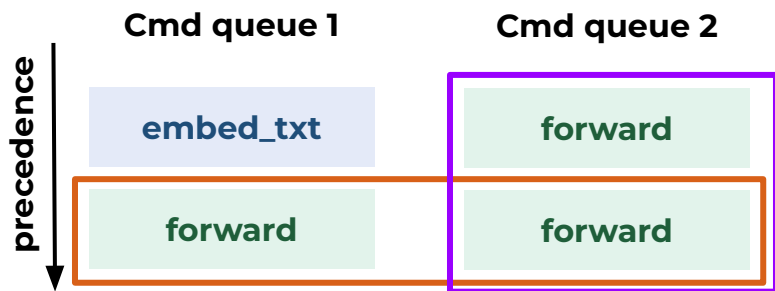
Control Layer



- **Controller** - handles API calls from concurrent inferlets
 - Handle non-GPU API calls, immediately
 - Managing allocation and virtual address mappings of Embed and KvPage resource (FCFS policy)
 - Sends GPU-bound calls to batch scheduler
- **Command Queue** - logical sequence of API calls issued by an inferlet
- **Batch Scheduler** - batches GPU-bound API calls before sending to inference layer
- **Event Dispatcher** - receives and broadcasts results from the inference layer back to the inferlets



Control Layer - Batching



- Two batching strategies:
 - **Vertical Batching**: consecutive same-type calls within one queue, batched if non-conflicting on KV pages
 - **Horizontal Batching**: same-type calls across different queues
- Dispatch policy: work-conserving, batch starts immediately when GPU is idle
- Within a batch: higher-priority queues go first, oldest pending call wins across API types



Inference Layer

Inference Layer

API Handler

API Handler

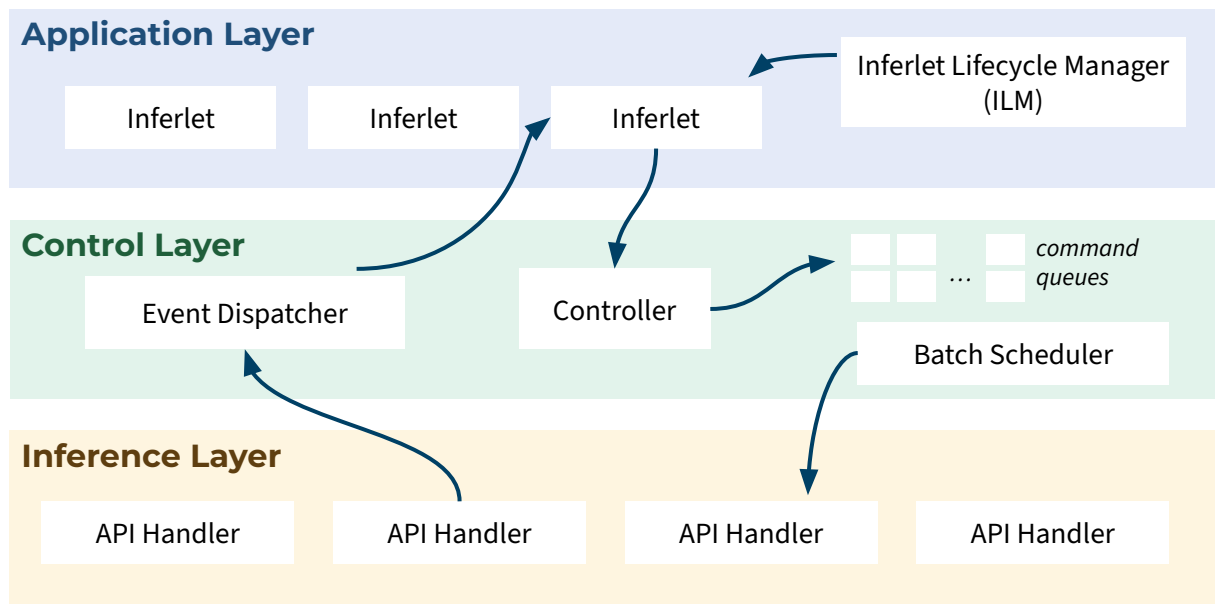
API Handler

API Handler

- GPU execution backend
- Receives batched API calls from the control layer via ZeroMQ IPC
- Specialized handlers execute each operation type on the GPU (PyTorch + FlashInfer)
- Results sent back to the control layer's Event Dispatcher, routed to the right inferlets
- Physical Embed and KvPage memory lives here, all management on Control layer
- Native C++/CUDA implementation achieves 10-30% lower latency vs. Python version



Putting It Together

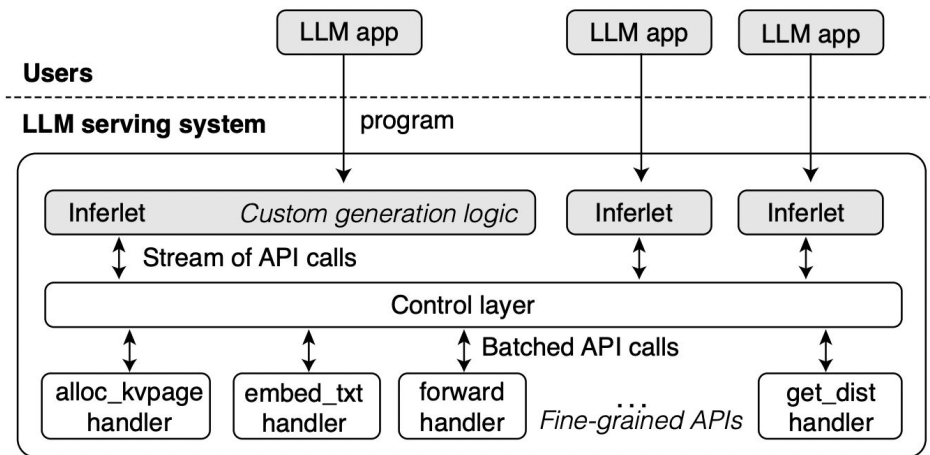
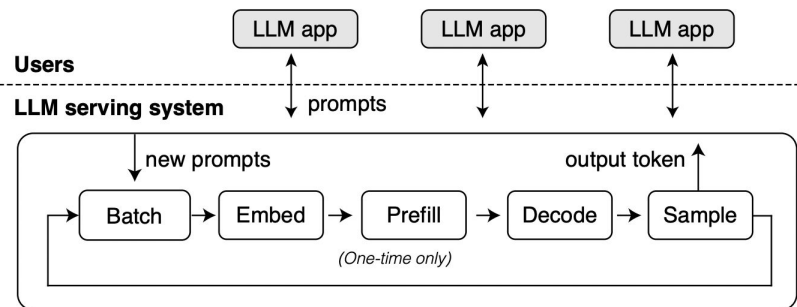




Differences from Related Work



Other Serving Systems vs Pie





Differences from Related Work

System	R1 KV Cache	R2 Generation	R3 I/O	Key Distinction
SGLang / Parrot <i>Application-aware serving</i>	~	~	✗	Introduce programmatic control constructs. Fork/join/gen (SGLang) and semantic variables (Parrot) partially address R1/R2 — but both remain fundamentally monolithic. No direct KV page control, no inline I/O. ★
LMQL / AICI / Guidance <i>Constrained decoding</i>	✗	~	✗	Control generation through grammars and constraints within standard decoding loop. Can't change how generation is performed. Pie extends this to control the full inference process: tokenization, attention, KV cache, sampling, and I/O.
LangChain / DSPy / AutoGen <i>Orchestration frameworks</i>	✗	✗	✗	Operate entirely above the serving layer with no access to inference internals. Complementary to Pie: these frameworks can use Pie as a backend and inherit its performance and I/O integration.
Pie <i>Inferlets, fine-grained handlers</i>	✓	✓	✓	Full control over embedding, attention, KV cache, sampling, and I/O — as user-provided programs, without modifying the serving system. The only system that addresses all three requirements.

Legend: ✓ Fully addressed

~ Partially addressed

✗ Not addressed

★ Parrot discussed in depth in joint section

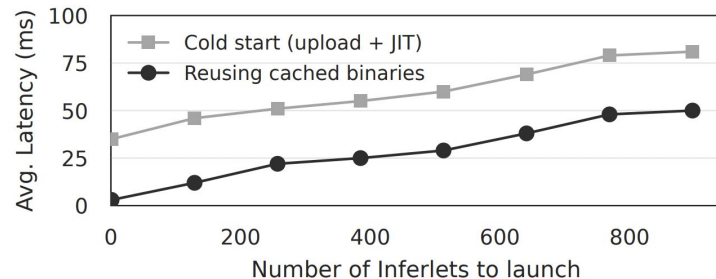
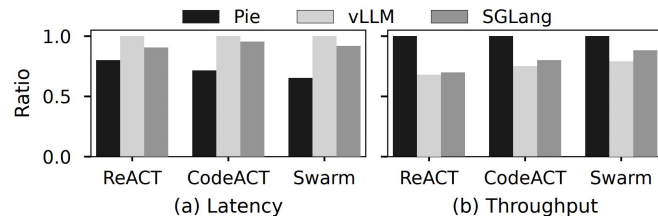


Strengths and Weaknesses



Strengths of Pie

- **Generality** - many techniques implemented as inferlets
 - Flexible serving model
 - Supports modern LLM workloads
 - Fine-grained control
- **Performance on complex workloads** - 1.1-2.4x lower latency, 1.3-3.4x higher throughput on agentic and reasoning tasks vs. baselines
- **Low standard-task overhead** - only 2-12% latency increase, shrinks on bigger 8B models
- **Extensibility** - Trait-based API means new capabilities just add a Trait, existing inferlets unchanged





Weaknesses of Pie

- **Programming complexity**
 - 42 APIs
 - users have to program their inferlets, memory has to be explicitly managed
 - Support library helps
- **Centralized control layer** - single
- **Narrow Evaluation** - no multi-GPU, no other model architectures
- **Security Surface** - inferlets can make HTTP calls and access raw token distributions, risks include DoS, side-channel leakage, model extraction
- **Resource Contention Policy** - FCFS termination



Directions for Future Work

- **Distributed Pie**
 - Add distributed scaling across multiple GPU nodes
 - Needs KV-locality-aware scheduling, distributed cache placement, fault tolerance
- **Higher-level abstractions**
 - Compiler that generates efficient inferlets from workflows to bridge programming usability gap
- **Improving Security**
- **Resource Contention Policies** - Authors call out per-inferlet quotas, priority-aware preemption, CPU-GPU KV swapping

Parrot vs. Pie



Category	Parrot	Pie
Goal	Improve end-to-end performance of LLM applications by exposing application-level structure to the serving system.	Provide programmable LLM serving infrastructure where applications fully control inference workflows.
Problem	Existing LLM services treat requests independently, losing workflow and prompt structure information.	Existing LLM systems rely on a fixed monolithic generation loop, limiting flexibility and application-specific optimization.
Implementation	Semantic Variables that annotate prompt structure and connect LLM calls.	Inferlets, user-defined programs controlling the generation process.
Programming Model	Developers define semantic functions using natural language prompts with placeholders (<code>{{input}}</code> , <code>{{output}}</code>).	Developers write inferlet programs (Wasm modules) orchestrating low-level inference APIs.
Dependency Handling	Automatically inferred using semantic variables and DAG analysis.	Explicitly managed by inferlet program logic.
Scheduling Strategy	Application-centric scheduling using DAG analysis and performance objectives.	API-call batching via command queues (horizontal and vertical batching).

Questions??